

INDEX

Exp. No	Experiment Name	Page no	Date	Signature
1	A menu driven program to perform operations on Array.	3	17/09/2025	
2	Merge two sorted array into a third array.	9	17/09/2025	
3	Stack implementation using Array.	11	17/09/2025	
4	Queue implementation using array	15	24/09/2025	
5	Circular Queue implementation using array	20	24/09/2025	
6	Operations on Singly Linked List	24	24/09/2025	
7	Operations on Binary Search Tree.	33	08/10/2025	
8	Operations on doubly linked list.	40	08/10/2025	
9	Operations on sets using Bit string implementation.	50	22/10/2025	
10	Perform Operations on Disjoint sets	56	22/10/2025	

Exp. No	Experiment Name	Page no	Date	Signature
11	Operations on linked stack.	61	22/10/2025	
12	Perform Breadth First Search(BFS) and Depth First Search(DFS) operations on graph.	67	12/11/2025	
13	Minimum Cost Spanning Tree using Prim's Algorithm.	73	12/11/2025	
14	Minimum cost spanning tree using Kruskal's Algorithm.	77	12/11/2025	

PROGRAM 1

AIM:

Create a menu driven program to perform operations (Create, Insert, Delete, Search) on Array.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
int choice,j,i=0,a[50],b[10],s;
void insert();
void delete();
void sort();
void search();
void display();
int menu();
int main()
{
    printf("\nEnter the size of array :");
    scanf("%d",&s);
    printf("\nEnter the elements of array :\n");
    for(i=0;i<s;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("\n---Array Operations---");
    printf("\n1.Insert");
    printf("\n2.Delete");
    printf("\n3.Sort");
    printf("\n4.Search");
    printf("\n5.Display");
```

```

printf("\n6.Exit\n");
while(1)
{
    printf("\nEnter your choice :");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1:insert();
        break;
        case 2:delete();
        break;
        case 3:sort();
        break;
        case 4:search();
        break;
        case 5:display();
        break;
        case 6:exit(1);
    }
}
return 0;
}

void search()
{
    int ele,loc,flag=0;
    printf("\nEnter the element to be searched :");
    scanf("%d",&ele);
    for(i=0;i<s;i++)
    {
        if(ele==a[i])
        {

```

```

        printf("Element found at :");
        printf("%d",i+1);
        flag=1;
        break;
    }
}
if(flag==0)
{
    printf("Element not found\n");
}
}

void sort()
{
    printf("\nCurrent array: ");
    for(i=0;i<s;i++)
        printf("\t%d",a[i]);
    for(i=0;i<s;i++)
    {
        for(j=i+1;j<s;j++)
        {
            if(a[i]>a[j])
            {
                a[i]=a[i]+a[j];
                a[j]=a[i]-a[j];
                a[i]=a[i]-a[j];
            }
        }
    }
    printf("\nsorted array: ");
    for(i=0;i<s;i++)
        printf("\t%d",a[i]);

```

```

}

void insert()
{
    int loc, ele;
    printf("\nEnter the location: ");
    scanf("%d", &loc);
    printf("Enter the element to be inserted: ");
    scanf("%d", &ele);
    if(loc < 1 || loc > s + 1) {
        printf("Invalid location!\n");
        return;
    }

    for(i = s; i >= loc; i--)
        a[i] = a[i - 1];
    a[loc - 1] = ele;
    s++;
    printf("Element inserted\nNew array after insertion:");
    for(i = 0; i < s; i++)
        printf("\t%d", a[i]);
}

void delete()
{
    int loc;
    printf("Enter the location: ");
    scanf("%d",&loc);
    if(loc<1 || loc>s)
        printf("Invalid location, Please enter a valid position\n");
    else
        {

```

```

        for(i=loc-1;i<s-1;i++)
            a[i]=a[i+1];
    }
    printf("Element deleted");
    printf("\nNew array after deletion: ");
    s--;
    for(i=0;i<s;i++)
        printf("\t%d",a[i]);
}

void display()
{
    int i;
    if(s==0)
    {
        printf("\nNo elements to display!");
        return;
    }
    printf("\nArray elements are: ");
    for(i=0;i<s;i++)
        printf("\t%d",a[i]);
}

```

OUTPUT:

```
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm1

Enter the size of array :4

Enter the elements of array :
4
8
6
2

---Array Operations---
1.Insert
2.Delete
3.Sort
4.Search
5.Display
6.Exit

Enter your choice :1

Enter the location: 5
Enter the element to be inserted: 3
Element inserted
New array after insertion:      4      8      6      2      3
Enter your choice :2
Enter the location: 4
Element deleted
New array after deletion:      4      8      6      3
Enter your choice :3

Current array:  4      8      6      3
sorted array:  3      4      6      8
Enter your choice :4

Enter the element to be searched :6
Element found at :3
Enter your choice :5

Array elements are:      3      4      6      8
Enter your choice :6
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny>
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 2**AIM:**

Merge two sorted array into a third array.

SOURCE CODE:

```
#include <stdio.h>

void merge(int a[], int b[], int c[], int m, int n);

int main()
{
    int m, n, i, a[20], b[20], c[40];

    printf("Enter the size of first array: ");
    scanf("%d", &m);
    printf("Enter the elements of first array: ");
    for (i = 0; i < m; i++)
        scanf("%d", &a[i]);
    printf("Enter the size of second array: ");
    scanf("%d", &n);
    printf("Enter the elements of second array: ");
    for (i = 0; i < n; i++)
        scanf("%d", &b[i]);
    merge(a, b, c, m, n);
    return 0;
}

void merge(int a[], int b[], int c[], int m, int n)
{
    int i = 0, j = 0, k = 0;
    while (i < m && j < n)
    {
        if (a[i] < b[j])
```

```

        c[k++] = a[i++];
    else
        c[k++] = b[j++];
}

while (i < m)
    c[k++] = a[i++];
while (j < n)
    c[k++] = b[j++];
printf("Merged array is:\n");
for (i = 0; i < k; i++)
    printf("%d ", c[i]);
printf("\n");
}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm2.c -o pgm2
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm2
Enter the size of first array: 5
Enter the elements of first array: 10
12
15
16
17
Enter the size of second array: 5
Enter the elements of second array: 21
22
25
36
47
Merged array is:
10 12 15 16 17 21 22 25 36 47

```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 3

AIM:

Stack implementation using array.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
void push(),pop(),display(),peek();
int stack[20],i,n,choice,item,top=-1;
int main()
{
    printf("Enter the size of stack:");
    scanf("%d",&n);
    printf("\n1.push\n2.pop\n3.display\n4.peek\n5.exit");
    do
    {
        printf("\nEnter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:push();
            break;
            case 2:pop();
            break;
            case 3:display();
            break;
            case 4:peek();
            break;
            case 5:exit(1);
        }
    }
```

```

        }while (choice != 5);
        return 0;
    }

void push()
{
    if(top!=n-1)
    {
        printf("Enter the element to be pushed:");
        scanf("%d",&item);
        top=top+1;
        stack[top]=item;
        printf("Inserted Successfully\n");
    }
    else
    {
        printf("Can't insert,stack is full!!\n");
    }
}

void pop()
{
    if(top==-1)
        printf("Can't delete data, stack is empty!");
    else
    {
        item=stack[top];
        printf("\nDeleted element:%d",item);
        top=top-1;
    }
}

```

```
void peek()
{
    if(top==-1)
        printf("No element in stack");
    else
        printf("The topmost element is:\t%d\n",stack[top]);
}

void display()
{
    int i=0;
    if(top==-1)
        printf("Stack is empty\n");
    else
    {
        printf("Stack elements are:");
        for(i=top;i>=0;i--)
            printf("%d\t",stack[i]);
    }
}
```

OUTPUT:

```
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm3.c -o pgm3
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm3
Enter the size of stack:5

1.push
2.pop
3.display
4.peek
5.exit
Enter your choice:4
No element in stack
Enter your choice:3
Stack is empty

Enter your choice:2
Can't delete data, stack is empty!
Enter your choice:1
Enter the element to be pushed:25
Inserted Successfully

Enter your choice:1
Enter the element to be pushed:35
Inserted Successfully

Enter your choice:3
Stack elements are:35    25
Enter your choice:2

Deleted element:35
Enter your choice:3
Stack elements are:25
Enter your choice:4
The topmost element is: 25

Enter your choice:5
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 4

AIM:

Queue implementation using array.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
void menu();
void enqueue();
void dequeue();
void display();
void peek();
int queue[50],i,n,choice,item,front=-1,rear=-1;
void main()
{
    printf("Enter the size of the queue:");
    scanf("%d",&n);
    menu();
}

void menu()
{
    int choice;
    printf("\n1.enqueue\n2.dequeue\n3.display\n4.peek\n5.exit");
    printf("\nEnter your choice:");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1:enqueue();
        break;
```

```

        case 2:dequeue();
        break;
        case 3:display();
        break;
        case 4:peek();
        break;
        case 5:exit(1);
    }
}

void enqueue()
{
    if(rear!=n-1)
    {
        printf("Enter the element to be inserted:");
        scanf("%d",&item);
        front=0;
        rear=rear+1;
        queue[rear]=item;
        printf("inserted successfully");
    }
    else
        printf("queue is full,can't insert data");
    menu();
}

void dequeue()
{
    if(front==-1 || front>rear)
        printf("queue is empty");
    else
    {

```



```

        item=queue[front];
        printf("deleted element:\t%d",item);
        if(front==rear)
            front=rear=-1;
        else
            front=front+1;
    }
    menu();
}

void display()
{
    int i=0;
    if(rear==-1)
        printf("queue is empty:");
    else
    {
        printf("queue elements are:");
        for(i=front;i<=rear;i++)
            printf("%d\t",queue[i]);
    }
    menu();
}

void peek()
{
    if(front==-1)
        printf("no element in queue\n");
    else
        printf("peek elemet:%d",queue[front]);
    menu();
}

```

OUTPUT:

```
PS C:\Users\gmrtto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm4.c -o pgm4
PS C:\Users\gmrtto\OneDrive\Desktop\ads\AlbinBenny> ./pgm4
Enter the size of the queue:5
```

```
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:4
no element in queue
```

```
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:3
queue is empty:
1.enqueue
2.dequeue
3.display
4.peek
5.exit
```

```
Enter your choice:2
queue is empty
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:1
Enter the element to be inserted:12
inserted successfully
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:1
Enter the element to be inserted:15
inserted successfully
1.enqueue
2.dequeue
3.display
4.peek
5.exit
```

```
Enter your choice:1
Enter the element to be inserted:25
inserted successfully
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:3
queue elements are:12    15    25
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:2
deleted element:    12
1.enqueue
2.dequeue
3.display
4.peek
```

```
Enter your choice:4
peek element:15
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:3
queue elements are:15    25
1.enqueue
2.dequeue
3.display
4.peek
5.exit
Enter your choice:5
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 5

AIM:

Circular Queue implementation using array.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
void enqueue(),dequeue(),display();
int queue[50],n,front=-1,rear=-1,choice,item,x;
int main()
{
    printf("Enter the size of the queue:");
    scanf("%d",&n);
    printf("\n1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
    while(1)
    {
        printf("Enter your choice:");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:enqueue();
            break;
            case 2:dequeue();
            break;
            case 3:display();
            break;
            case 4:exit(0);
        }
    }
    return 0;
```

```

}

void enqueue()
{
    if((front==rear+1) || (front==0 && rear==n-1))
        printf("\nOverflow\n");
    else
    {
        if(front==-1)
            front=0;
        rear=(rear+1)%n;
        printf("\nEnter element:");
        scanf("%d",&x);
        queue[rear]=x;
        printf("\nInserted element is %d\n",x);
    }
}

void dequeue()
{
    if(front==-1)
        printf("\nUnderflow\n");
    else
    {
        x=queue[front];
        if(front==rear)
            front=rear=-1;
        else
            front=(front+1)%n;
        printf("\nDeleted element is %d\n",x);
    }
}

```

```

void display()
{
    int i;
    if(front==-1)
        printf("\nEmpty queue\n");
    else
    {
        printf("Queue elements are:");
        for(i=front;i!= rear;i=(i+1)%n)
            printf("\t%d",queue[i]);
        printf("\t%d",queue[rear]);
        printf("\n");
    }
}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm5.c -o pgm5
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm5
Enter the size of the queue:5

1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice:3

Empty queue
Enter your choice:2

Underflow
Enter your choice:1

Enter element:12

Inserted element is 12
Enter your choice:1

Enter element:32

Inserted element is 32
Enter your choice:1

Enter element:10

```

```
Inserted element is 10
Enter your choice:1

Enter element:14

Inserted element is 14
Enter your choice:1

Enter element:68

Inserted element is 68
Enter your choice:1

Overflow
Enter your choice:3
Queue elements are:      12      32      10      14      68
Enter your choice:2

Deleted element is 12
Enter your choice:2

Deleted element is 32
Enter your choice:3
Queue elements are:      10      14      68
Enter your choice:4
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 6

AIM:

Operations on Singly Linked List.

SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

void display(),in_e(),in_bw(),del_b(),del_e(),del_bw(),menu();

int in_b();

int y,n;

char i;

struct node *create();

struct node
{
    int data;
    struct node *link;
};

typedef struct node node1;

node1 *start=NULL;

int main()
{
    printf("Singly linked list\n");
    printf("MENU\n");
    printf("\n1.Display\n2.Insertion at beginning\n3.Insertion at end\n4.Insertion in
    between\n5.Deletion at beginning\n6.deletion at end\n7.Deletion in between\n8.Exit\n");
    menu();
}

void menu()
{
```



```

int x;

printf("\nEnter your choice:");

scanf("%d",&x);

switch(x)
{
    case 1:display();
    break;
    case 2:in_b();
    break;
    case 3:in_e();
    break;
    case 4:in_bw();
    break;
    case 5:del_b();
    break;
    case 6:del_e();
    break;
    case 7:del_bw();
    break;
    case 8:exit(0);

}
}

struct node *create()
{
    node1 *nptr=((node1*)malloc(sizeof(node1)));
    if(nptr==NULL)
    {
        printf("Memory Overflow");
        return 0;
    }
    else

```

```

        return nptr;
    }

void display()
{
    node1 *ptr=start;
    if(ptr==NULL)
        printf("\nNo elements to display\n");
    else
    {
        printf("\nElements in the linked list are:\n");
        while(ptr!=NULL)
        {
            printf("%d\t",ptr->data);
            ptr=ptr->link;
        }
    }
    menu();
}

int in_b()
{
    int val;
    node1 *nptr;
    printf("\nEnter the element to be inserted:");
    scanf("%d",&val);
    nptr=create();
    nptr->data=val;
    if(start==NULL)
    {
        start=nptr;
        nptr->link=NULL;
    }
}

```

```

    }
    else
    {
        nptr->link=start;
        start=nptr;
    }
    printf("Element inserted\n");
    menu();
}

void in_e()
{
    node1 *nptr=create(),*temp;
    int val;
    printf("\nEnter the element to be inserted:");
    scanf("%d",&val);
    nptr->data=val;
    nptr->link=NULL;
    temp=start;
    while(temp->link!=NULL)
        temp=temp->link;
    temp->link=nptr;
    printf("Element inserted\n");
    menu();
}

void in_bw()
{
    node1 *temp,*nptr=create();
    int val,pos,i;
    printf("\nEnter the element to be inserted:");
    scanf("%d",&val);

```

```

    nptr->data=val;
    nptr->link=NULL;
    temp=start;
    printf("\nEnter the position to be inserted:");
    scanf("%d",&pos);
    if(pos==1)
    {
        nptr->link=start;
        start=nptr;
    }
    else
    {
        for(i=1;i<pos-1;i++)
            temp=temp->link;
    }
    nptr->link=temp->link;
    temp->link=nptr;
    printf("Element inserted\n");
    menu();
}

```

```

void del_b()
{
    node1 *temp;
    if(start==NULL)
        printf("\nList is empty!");
    else
    {
        temp=start;
        start=start->link;
        free(temp);
        printf("\nElement deleted\n");
    }
}

```

```

    }
    menu();
}

void del_e()
{
    node1 *temp,*prev;
    temp=start;
    if(start==NULL)
        printf("List is empty!");
    while(temp->link!=NULL)
    {
        prev=temp;
        temp=temp->link;
    }
    prev->link=NULL;
    free(temp);
    printf("\nElement deleted\n");
    menu();
}

void del_bw()
{
    node1 *temp,*pre;
    int pos,i;
    printf("Enter the position:");
    scanf("%d",&pos);
    temp=start;
    if(start==NULL)
        printf("List is empty!");
    if(pos==1)
        start=start->link;

```

```

else
{
    for (i=1;i<=pos;i++)
    {
        pre=temp;
        temp=temp->link;
        pre->link=temp->link;
    }
}
printf("\nElement deleted\n");
menu();
}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm6.c -o pgm6
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm6
Singly linked list
MENU

1.Display
2.Insertion at beginning
3.Insertion at end
4.Insertion in between
5.Deletion at beginning
6.deletion at end
7.Deletion in between
8.Exit

Enter your choice:1

No elements to display

Enter your choice:2

Enter the element to be inserted:147
Element inserted

Enter your choice:2

Enter the element to be inserted:156
Element inserted

Enter your choice:2

Enter the element to be inserted:123
Element inserted

```

Enter your choice:1

Elements in the linked list are:

123 156 147

Enter your choice:3

Enter the element to be inserted:256

Element inserted

Enter your choice:1

Elements in the linked list are:

123 156 147 256

Enter your choice:4

Enter the element to be inserted:258

Enter the position to be inserted:3

Element inserted

Enter your choice:1

Elements in the linked list are:

123 156 258 147 256

Enter your choice:5

Element deleted

Enter your choice:1

Elements in the linked list are:

156 258 147 256

Enter your choice:6

Element deleted

Enter your choice:1

Elements in the linked list are:

156 258 147

Enter your choice:7

Enter the position:2

Element deleted

Enter your choice:1

Elements in the linked list are:

156 147

Enter your choice:8

RESULT:

Program executed and Output obtained successfully.

PROGRAM 7**AIM:**

Operations on Binary Search Tree.

SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

struct btnode {
    int value;
    struct btnode *l;
    struct btnode *r;
} *root = NULL, *temp = NULL, *t1 = NULL, *tl = NULL;

// Function declarations
void insert();
void create();
void search(struct btnode *t);
void inorder(struct btnode *t);
void delete1();
void search1(struct btnode *t, int data);
void deletel(struct btnode *t);
int smallest(struct btnode *t);

int main() {
    int ch;

    printf("\n---- BST OPERATIONS ----");

    printf("\n1 - Insert an element");

    printf("\n2 - Delete an element");

    printf("\n3 - Inorder Traversal");
```

```

printf("\n4 - Exit");

while (1) {
    printf("\n\nEnter your choice: ");
    scanf("%d", &ch);
    switch (ch) {
        case 1:
            insert();
            break;
        case 2:
            delete1();
            break;
        case 3:
            if (root == NULL)
                printf("Tree is empty");
            else
                inorder(root);
            break;
        case 4:
            exit(0);
        default:
            printf("Wrong choice! Try again.");
    }
}

return 0;
}

void insert() {
    create();
    if (root == NULL)
        root = temp;
    else

```

```

        search(root);
    }

void create() {
    int data;
    printf("Enter data of node to be inserted: ");
    scanf("%d", &data);
    temp = (struct btnode *)malloc(sizeof(struct btnode));
    temp->value = data;
    temp->l = temp->r = NULL;
}

void search(struct btnode *t) {
    if (temp->value < t->value) {
        if (t->l == NULL)
            t->l = temp;
        else
            search(t->l);
    } else if (temp->value > t->value) {
        if (t->r == NULL)
            t->r = temp;
        else
            search(t->r);
    } else {
        printf("Duplicate value not allowed!\n");
    }
}

void inorder(struct btnode *t) {
    if (t == NULL)
        return;
    inorder(t->l);

```

```

    printf("%d -> ", t->value);
    inorder(t->r);
}

void deletel() {
    int data;
    if (root == NULL) {
        printf("No elements in the tree to delete");
        return;
    }

    printf("Enter the data to be deleted: ");
    scanf("%d", &data);
    t1 = NULL;
    tl = NULL;

    search1(root, data);
    if (t1 == NULL)
        printf("Element not found in the tree");
    else
        deletel(t1);
}

void search1(struct btnode *t, int data) {
    if (t == NULL)
        return;

    if (data < t->value) {
        tl = t;
        search1(t->l, data);
    } else if (data > t->value) {
        tl = t;

```

```

        searchl(t->r, data);
    } else {
        t1 = t; // Node found
    }
}

void deletel(struct btnode *t) {
    int k;

    // Case 1: No child
    if (t->l == NULL && t->r == NULL) {
        if (t == root)
            root = NULL;
        else if (t1->l == t)
            t1->l = NULL;
        else
            t1->r = NULL;
        free(t);
        return;
    }

    // Case 2: Only left child
    if (t->r == NULL) {
        if (t == root)
            root = t->l;
        else if (t1->l == t)
            t1->l = t->l;
        else
            t1->r = t->l;
        free(t);
        return;
    }
}

```

```

// Case 3: Only right child
if (t->l == NULL) {
    if (t == root)
        root = t->r;
    else if (tl->r == t)
        tl->r = t->r;
    else
        tl->l = t->r;
    free(t);
    return;
}

// Case 4: Two children
k = smallest(t->r);
t->value = k;
t1 = t;
search1(t->r, k);
deletel(t1);
}

int smallest(struct bnode *t) {
    if (t->l == NULL)
        return t->value;
    else
        return smallest(t->l);
}

```

OUTPUT:

```
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm7.c -o pgm7
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm7
```

```
---- BST OPERATIONS ----
```

- 1 - Insert an element
- 2 - Delete an element
- 3 - Inorder Traversal
- 4 - Exit

```
Enter your choice: 1
Enter data of node to be inserted: 14
```

```
Enter your choice: 1
Enter data of node to be inserted: 15
```

```
Enter your choice: 1
Enter data of node to be inserted: 33
```

```
Enter your choice: 1
Enter data of node to be inserted: 25
```

```
Enter your choice: 1
Enter data of node to be inserted: 20
```

```
Enter your choice: 3
14 -> 15 -> 20 -> 25 -> 33 ->
```

```
Enter your choice: 2
Enter the data to be deleted: 21
Element not found in the tree
```

```
Enter your choice: 2
Enter the data to be deleted: 20
```

```
Enter your choice: 3
14 -> 15 -> 25 -> 33 ->
```

```
Enter your choice: 4
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 8**AIM:**

Operations on doubly linked list.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL, *t = NULL;
struct node *newnode, *d;
int count = 0;

void create()
{
    int v;
    newnode = (struct node *)malloc(sizeof(struct node));
    if(newnode == NULL)
    {
        printf("Memory allocation failed");
        exit(1);
    }

    printf("Enter the value to insert:\n");
```



```

scanf("%d", &v);

newnode->data = v;
newnode->prev = NULL;
newnode->next = NULL;
}

void ins_beg()
{
    create();
    if(head == NULL)
    {
        head = newnode;
    }
    else
    {
        newnode->next = head;
        head->prev = newnode;
        head = newnode;
    }
    printf("\nNode inserted at beginning\n");
    count++;
}

void ins_end()
{
    create();
    if(head == NULL)
    {
        head = newnode;
    }
    else

```

```

{
    t = head;
    while(t->next != NULL)
    {
        t = t->next;
    }
    t->next = newnode;
    newnode->prev = t;
}
printf("\nNode inserted at end\n");
count++;
}

void ins_pos()
{
    int pos, i;
    printf("Enter the position to insert:");
    scanf("%d", &pos);

    if(pos == 1)
    {
        ins_beg();
    }
    else if(pos == count + 1)
    {
        ins_end();
    }
    else if(pos > 0 && pos <= count)
    {
        create();
        t = head;
        for(i = 1; i < pos - 1; i++)

```

```

    {
        t = t->next;
    }

    newnode->next = t->next;
    newnode->prev = t;
    t->next->prev = newnode;
    t->next = newnode;

    printf("\nNode inserted at position %d\n", pos);
    count++;
}
else
{
    printf("Invalid position\n");
}
}

void del_beg()
{
    if(head == NULL)
    {
        printf("\nLinked list is empty\n");
    }
    else if(head->next == NULL)
    {
        d = head;
        head = NULL;
        free(d);
        printf("\nNode deleted from beginning\n");
        count--;
    }
}

```

```

else
{
    d = head;
    head = head->next;
    head->prev = NULL;
    free(d);
    printf("\nNode deleted from beginning\n");
    count--;
}
}

```

```

void del_end()
{
    if(head == NULL)
    {
        printf("\nLinked list is empty\n");
    }
    else if(head->next == NULL)
    {
        d = head;
        head = NULL;
        free(d);
        printf("\nNode deleted from end\n");
        count--;
    }
    else
    {
        d = head;
        while(d->next != NULL)
        {
            d = d->next;
        }
    }
}

```

```

    d->prev->next = NULL;

    free(d);

    printf("\nNode deleted from end\n");

    count--;

}

}

void del_pos()
{
    int pos, i;

    printf("Enter the position to delete:");

    scanf("%d", &pos);

    if(head == NULL)
    {
        printf("\nLinked list is empty\n");
    }
    else if(pos == 1)
    {
        del_beg();
    }
    else if(pos == count)
    {
        del_end();
    }
    else if(pos > 1 && pos < count)
    {
        d = head;

        for(i = 1; i < pos; i++)
        {
            d = d->next;
        }
    }
}

```

```

    d->prev->next = d->next;
    d->next->prev = d->prev;

    free(d);
    printf("\nNode deleted from position %d\n", pos);
    count--;
}
else
{
    printf("\nInvalid position\n");
}
}

void display()
{
    if(head == NULL)
    {
        printf("\nLinked list is empty\n");
    }
    else
    {
        printf("\nLinked list is:\n");
        t = head;
        while(t != NULL)
        {
            printf("%d\t", t->data);
            t = t->next;
        }
        printf("\n");
    }
}

```

```

int main()
{
    int ch;

    printf("\n***** DOUBLY LINKED LIST *****\n");
    printf("\n1. Insert at beginning");
    printf("\n2. Insert at end");
    printf("\n3. Insert at a particular position");
    printf("\n4. Delete from beginning");
    printf("\n5. Delete from end");
    printf("\n6. Delete from a particular position");
    printf("\n7. Display");
    printf("\n8. Exit");

    while(1)
    {
        printf("\n\nEnter your choice: ");
        scanf("%d", &ch);

        switch(ch)
        {
            case 1: ins_beg(); break;
            case 2: ins_end(); break;
            case 3: ins_pos(); break;
            case 4: del_beg(); break;
            case 5: del_end(); break;
            case 6: del_pos(); break;
            case 7: display(); break;
            case 8:
                printf("\nExiting program...\n");
                exit(0);
            default:

```

```

        printf("\nInvalid choice\n");

    }

}

return 0;

}

```

OUTPUT:

```

PS C:\Users\gmerto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm8.c -o pgm8
PS C:\Users\gmerto\OneDrive\Desktop\ads\AlbinBenny> ./pgm8

```

```

***** DOUBLY LINKED LIST *****

1. Insert at beginning
2. Insert at end
3. Insert at a particular position
4. Delete from beginning
5. Delete from end
6. Delete from a particular position
7. Display
8. Exit

```

```

Enter your choice: 1
Enter the value to insert:
10

```

Node inserted at beginning

```

Enter your choice: 1
Enter the value to insert:
14

```

Node inserted at beginning

```

Enter your choice: 1
Enter the value to insert:
18

```

Node inserted at beginning

```

Enter your choice: 7

```

```

Linked list is:
18      14      10

```

```

Enter your choice: 2
Enter the value to insert:
36

```



```
Node inserted at end

Enter your choice: 7

Linked list is:
18      14      10      36

Enter your choice: 3
Enter the position to insert:3
Enter the value to insert:
27

Node inserted at position 3

Enter your choice: 7

Linked list is:
18      14      27      10      36

Enter your choice: 6
Enter the position to delete:4

Node deleted from position 4

Enter your choice: 7

Linked list is:
18      14      27      36
```

```
Enter your choice: 4

Node deleted from beginning

Enter your choice: 7

Linked list is:
14      27      36

Enter your choice: 5

Node deleted from end

Enter your choice: 7

Linked list is:
14      27

Enter your choice: 8

Exiting program...
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 9

AIM:

Operations on sets using Bit string implementation.

SOURCE CODE:

```
#include <stdio.h>

void input();
void output();
void setunion();
void intersection();
void difference();

int a[]={0,0,0,0,0,0,0,0},b[]={0,0,0,0,0,0,0,0};

int main()
{
    int ch,wish;
    do
    {
        printf("\n MENU \n");
        printf("1.Input\n2.Union\n3.Intersection\n4.Difference\n");
        printf("Enter choice\n");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:input();
                break;
            case 2:setunion();
                break;
            case 3:intersection();
                break;
            case 4:difference();
```

```

        break;

    }

    printf("\nDo you wish to continue ?(1/0)\n");
    scanf("%d",&wish);
}
while(wish==1);
}

void input()
{
    int n,x,i;
    printf("enter size of the 1st set\n");
    scanf("%d",&n);
    printf("enter elements\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&x);
        a[x-1]=1;
    }
    printf("enter size of the 2nd set\n");
    scanf("%d",&n);
    printf("enter elements\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&x);
        b[x-1]=1;
    }
    printf("\n1st set\n");
    for(i=0;i<9;i++)
    {
        printf("%d",a[i]);
    }
}

```

```

        printf("\n2nd set\n");
        for(i=0;i<9;i++)
        {
            printf("%d",b[i]);
        }
    }

void output(int c[])
{
    int i;
    printf("\n Set is ");
    for(i=0;i<9;i++)
    {
        if (c[i]!=0)
            printf("%d ",i+1);
    }
}

void setunion()
{
    int i,c[10];
    for(i=0;i<9;i++)
    {
        if (a[i]!=b[i])
            c[i]=1;
        else
            c[i]=a[i];
    }
    for(i=0;i<9;i++)
    {
        printf("%d",c[i]);
    }
}

```

```

        output(c);
    }

void intersection()
{
    int i,c[10];
    for(i=0;i<9;i++)
    {
        if (a[i]==b[i])
            c[i]=a[i];
        else
            c[i]=0;
    }
    for(i=0;i<9;i++)
    {
        printf("%d",c[i]);
    }
    output(c);
}

void difference()
{
    int i,c[10];
    for(i=0;i<9;i++)
    {
        if (a[i]==1 && b[i]==0)
            c[i]=1;
        else
            c[i]=0;
    }
    for(i=0;i<9;i++)
    {

```

```

        printf("%d",c[i]);

    }

    output(c);
}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm9.c -o pgm9
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm9

```

```

MENU
1.Input
2.Union
3.Intersection
4.Difference
Enter choice
1
enter size of the 1st set
4
enter elements
2
7
4
9
enter size of the 2nd set
3
enter elements
6
5
9

1st set
010100101
2nd set
000011001
Do you wish to continue ?(1/0)
1

MENU
1.Input
2.Union
3.Intersection
4.Difference
Enter choice
2
010111101
Set is 2 4 5 6 7 9

```

```
Do you wish to continue ?(1/0)
1

MENU
1.Input
2.Union
3.Intersection
4.Difference
Enter choice
3
000000001
Set is 9
Do you wish to continue ?(1/0)
1

MENU
1.Input
2.Union
3.Intersection
4.Difference
Enter choice
4
010100100
Set is 2 4 7
Do you wish to continue ?(1/0)
0
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 10**AIM:**

Perform Operations on Disjoint sets.

SOURCE CODE:

```
#include<stdio.h>

struct DisjSet{
int parent[10];
int rank[10];
int n;
}dis;

void makeSet()
{
    int i;
    for( i=0;i<dis.n;i++)
    {
        dis.parent[i]=i;
        dis.rank[i]=0;
    }
}

void displaySet()
{
    int i;
    printf("\nParent Array:\n");
    for( i=0;i<dis.n;i++)
    {
        printf("\t%d",dis.parent[i]);
    }
    printf("\nRank Array:\n");
    for(int i=0;i<dis.n;i++)
```



```

    {
        printf("\t%d",dis.rank[i]);
    }
    printf("\n");
}

int find(int x)
{
    if(dis.parent[x]!=x)
    {
        dis.parent[x]=find(dis.parent[x]);
    }
    return dis.parent[x];
}

void Union(int x,int y)
{
    int xset=find(x);
    int yset=find(y);
    if(xset==yset)
        return;
    if(dis.rank[xset]<dis.rank[yset])
    {
        dis.parent[xset]=yset;
        dis.rank[xset]=-1;
    }
    else if(dis.rank[xset]>dis.rank[yset])
    {
        dis.parent[yset]=xset; dis.rank[yset]=-1;
    }
    else
    {

```

```

        dis.parent[yset]=xset;
        dis.rank[xset]=dis.rank[xset] + 1;
        dis.rank[yset]=-1;
    }
}

int main()
{
    int n,x,y;
    printf("\nHow many elements:");
    scanf("%d",&dis.n);
    makeSet();
    int ch,wish;
    do
    {
        printf("\nMENU\n");
        printf("1.Union\n2.Find\n3.Display\nEnter choice:");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
                printf("\nEnter elements to perform union:");
                scanf("%d %d",&x,&y);
                Union(x,y);
                break;
            case 2:
                printf("Enter elements to check if connected components:");
                scanf("%d %d",&x,&y);
                if(find(x)==find(y))
                    printf("Connected components\n");
                else
                    printf("Not connected components\n");
            case 3:
                Display();
                break;
        }
    } while(ch!=0);
}

```

```

        break;

        case 3:

            displaySet();

            break;

    }

    printf("\nDo you wish to continue?(1/0)\n");

    scanf("%d",&wish);

}

while(wish==1);

return 0;

}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm10.c -o pgm10
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm10

```

How many elements:5

MENU

1.Union

2.Find

3.Display

Enter choice:3

Parent Array:

0	1	2	3	4
---	---	---	---	---

Rank Array:

0	0	0	0	0
---	---	---	---	---

Do you wish to continue?(1/0)

1

MENU

1.Union

2.Find

3.Display

Enter choice:1

Enter elements to perform union:2 3

Do you wish to continue?(1/0)

1

MENU

1.Union

2.Find

3.Display

Enter choice:3

```

Parent Array:
    0      1      2      2      4
Rank Array:
    0      0      1     -1      0

Do you wish to continue?(1/0)
1

MENU
1.Union
2.Find
3.Display
Enter choice:2
Enter elements to check if connected components:2 3
Connected components

Do you wish to continue?(1/0)
1

MENU
1.Union
2.Find
3.Display

Enter choice:2
Enter elements to check if connected components:2 4
Not connected components

Do you wish to continue?(1/0)
0

```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 11**AIM:**

Operations on linked stack.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>

void push();
void pop();
void display();

struct node
{
    int val;
    struct node *next;
};

struct node *start;

void main()
{
    int choice=0;
    printf("\nStack operations using linked list\n");
    while(choice!=4)
    {
        printf("\n---Menu---\n");
        printf("\n1.Push\n2.Pop\n3.Display\n4.Exit");
        printf("\nEnter your choice:\n");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
```

```

        {
            push();
            break;
        }
    case 2:
        {
            pop();
            break;
        }
    case 3:
        {
            display();
            break;
        }
    case 4:
        {
            printf("Existing...");
            break;
        }
    default:
        {
            printf("please Enter Valid choice");
        }
};
}

```

void push()

```

{
    int val;
    struct node*ptr=(struct node*)malloc(sizeof(struct node));
    if(ptr==NULL)

```

```

        printf("not able to push the element");

    else
    {
        printf("enter the value");
        scanf("%d",&val);
        if(start==NULL)
        {
            ptr->val=val;
            ptr->next=NULL;
            start=ptr;
        }
        else
        {
            ptr->val=val;
            ptr->next=start;
            start=ptr;
        }
        printf("item pushed");
    }
}

```

```

void pop()
{
    int item;
    struct node*ptr;
    if(start==NULL)
        printf("underflow");
    else
    {
        item=start->val;
        ptr=start;
        start=start->next;
    }
}

```

```

        free(ptr);
        printf("item popped");
    }
}

void display()
{
    int i;
    struct node *ptr;
    ptr=start;
    if(ptr==NULL)
        printf("Stack is empty\n");
    else
    {
        printf("Printing Stack elements\n");
        while(ptr!=NULL)
        {
            printf("%d\n",ptr->val);
            ptr=ptr->next;
        }
    }
}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm11.c -o pgm11
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm11

Stack operations using linked list

---Menu---

1.Push
2.Pop
3.Display
4.Exit
Enter your choice:
3
Stack is empty

```


---Menu---

1.Push

2.Pop

3.Display

4.Exit

Enter your choice:

2

underflow

---Menu---

1.Push

2.Pop

3.Display

4.Exit

Enter your choice:

1

enter the value2

item pushed

---Menu---

1.Push

2.Pop

3.Display

4.Exit

Enter your choice:

1

enter the value3

item pushed

---Menu---

1.Push

2.Pop

3.Display

4.Exit

Enter your choice:

1

enter the value4

item pushed

---Menu---

1.Push

2.Pop

3.Display

4.Exit

Enter your choice:

1

```
enter the value5
item pushed
---Menu---

1.Push
2.Pop
3.Display
4.Exit
Enter your choice:
3
Printing Stack elements
5
4
3
2

---Menu---

1.Push
2.Pop
3.Display
4.Exit
Enter your choice:
2
item popped
---Menu---

1.Push
2.Pop
3.Display
4.Exit
Enter your choice:
3
Printing Stack elements
4
3
2
```

```
---Menu---

1.Push
2.Pop
3.Display
4.Exit
Enter your choice:
4
Existing...
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 12**AIM:**

Perform Breadth First Search(BFS) and Depth First Search(DFS) operations on graph.

SOURCE CODE:

```
#include <stdio.h>

int q[20], top = -1, front = -1, rear = -1, a[20][20], vis[20], stack[20];

void add(int item);

void bfs(int s, int n);

void dfs(int s, int n);

void push(int item);

int pop();

int delete1();

int main()
{
    int n, i, s, ch, j;
    char c, dummy;
    printf("Enter the number of vertices:");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("[%d] [%d] ", i, j);
            scanf("%d", &a[i][j]);
        }
    }
    printf("The adjacency matrix is:\n");
    for (i = 1; i <= n; i++)
    {
```

```

        for (j = 1; j <= n; j++)
        {
            printf("%d ", a[i][j]);

        }
        printf("\n");
    }
do
{
    for (i = 1; i <= n; i++)
        vis[i] = 0;

    printf("\nMENU");
    printf("\n1.B.F.S");
    printf("\n2.D.F.S");
    printf("\nEnter your choice:");
    scanf("%d", &ch);
    printf("Enter the source vertex :");
    scanf("%d", &s);
    switch(ch)
    {
        case 1:
            bfs(s, n);
            break;
        case 2:
            dfs(s, n);
            break;
    }
    printf("\nDo you want to continue(Y/N) ? ");
    scanf("%c", &dummy);
    scanf("%c", &c);
}
while ((c == 'y') || (c == 'Y'));
return 0;

```

```

}

void bfs(int s, int n)
{
    int p, i;
    add(s);
    vis[s] = 1;
    p = delete1();
    if (p != 0)
        printf("%d ", p);
    while (p != 0)
    {
        for (i = 1; i <= n; i++)
        {
            if ((a[p][i] != 0) && (vis[i] == 0))
            {
                add(i);
                vis[i] = 1;
            }
        }
        p = delete1();
        if (p != 0)
            printf("%d ", p);
    }
    for (i = 1; i <= n; i++)
    {
        if (vis[i] == 0)
            bfs(i, n);
    }
}

void add(int item)

```

```

{
    if (rear == 19)
        printf("queue full");
    else
    {
        if (rear == -1)
        {
            q[++rear] = item;
            front++;
        }
        else
            q[++rear] = item;
    }
}

```

```

int delete1()

```

```

{
    int k;
    if ((front > rear) || (front == -1))
        return (0);
    else
    {
        k = q[front++];
        return (k);
    }
}

```

```

void dfs(int s, int n)

```

```

{
    int i, k;
    push(s);
    vis[s] = 1;

```

```

k = pop();
if (k != 0)
    printf("%d ", k);
while (k != 0)
{
    for (i = 1; i <= n; i++)
    {
        if ((a[k][i] != 0) && (vis[i] == 0))
        {
            push(i);
            vis[i] = 1;
        }
    }
    k = pop();
    if (k != 0)
        printf("%d ", k);
}
for (i = 1; i <= n; i++)
{
    if (vis[i] == 0)
        dfs(i, n);
}
}

```

```

void push(int item)
{
    if (top == 19)
        printf("stack overflow");
    else
        stack[++top] = item;
}

```

```

int pop()
{
    int k;
    if (top == -1)
        return (0);
    else
    {
        k = stack[top--];
        return (k);
    }
}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm12.c -o pgm12
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm12
Enter the number of vertices:3
[1] [1] 0
[1] [2] 2
[1] [3] 5
[2] [1] 3
[2] [2] 0
[2] [3] 4
[3] [1] 1
[3] [2] 6
[3] [3] 0
The adjacency matrix is:
0 2 5
3 0 4
1 6 0

MENU
1.B.F.S
2.D.F.S
Enter your choice:1
Enter the source vertex :2
2 1 3
Do you want to continue(Y/N) ? y

MENU
1.B.F.S
2.D.F.S
Enter your choice:2
Enter the source vertex :2
2 3 1
Do you want to continue(Y/N) ? n

```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 13**AIM:**

Minimum Cost Spanning Tree using Prim's Algorithm.

SOURCE CODE:

```
#include <stdio.h>

#define MAX 10

int graph[MAX][MAX], parent[MAX], key[MAX];
int n;

int minKey(int key[], int mstSet[])
{
    int min = 999, min_index;
    for (int v = 0; v < n; v++)
    {
        if (mstSet[v] == 0 && key[v] < min)
        {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void printMST()
{
    printf("Edges of Minimum Spanning Tree:\n");
    for (int i = 1; i < n; i++)
    {
        printf("Edge: %d - %d, Weight: %d\n", parent[i]+1, i+1, graph[i][parent[i]]);
    }
}
```

```

}

void primMST()
{
    int mstSet[MAX];
    for (int i = 0; i < n; i++)
    {
        key[i] = 999;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < n - 1; count++)
    {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;
        for (int v = 0; v < n; v++)
        {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v])
            {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
}

int calculateTotalCost()
{
    int totalCost = 0;
    for (int i = 1; i < n; i++)
    {

```

```

        totalCost += graph[i][parent[i]];

    }

    return totalCost;
}

int main()
{
    printf("\n\tImplementation of Prim's Algorithm\n");
    printf("\nEnter the no. of vertices:");
    scanf("%d", &n);
    printf("\nEnter the cost adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("[%d] [%d]: ", i+1, j+1);
            scanf("%d", &graph[i][j]);
        }
    }

    primMST();
    printf("\nTotal cost of Minimum Spanning Tree: %d\n", calculateTotalCost());
    printMST();
    return 0;
}

```

OUTPUT:

```
PS C:\Users\gmerto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm13.c -o pgm13
PS C:\Users\gmerto\OneDrive\Desktop\ads\AlbinBenny> ./pgm13
```

Implementation of Prim's Algorithm

Enter the no. of vertices:4

Enter the cost adjacency matrix:

```
[1] [1]: 0
[1] [2]: 1
[1] [3]: 2
[1] [4]: 3
[2] [1]: 1
[2] [2]: 0
[2] [3]: 4
[2] [4]: 5
[3] [1]: 2
[3] [2]: 4
[3] [3]: 0
[3] [4]: 6
[4] [1]: 3
[4] [2]: 5
[4] [3]: 6
[4] [4]: 0
```

Total cost of Minimum Spanning Tree: 6

Edges of Minimum Spanning Tree:

Edge: 1 - 2, Weight: 1

Edge: 1 - 3, Weight: 2

Edge: 1 - 4, Weight: 3

```
PS C:\Users\gmerto\OneDrive\Desktop\ads\AlbinBenny> |
```

RESULT:

Program executed and Output obtained successfully.

PROGRAM 14**AIM:**

Minimum cost spanning tree using Kruskal's Algorithm.

SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

int i, j, k, a, b, u, v, n, ne = 1;

int min, mincost = 0, cost[9][9], parent[9];

int find(int);

int uni(int, int);

int main()
{
    printf("\n\tImplementation of Kruskal's Algorithm\n");
    printf("\nEnter the no. of vertices:");
    scanf("%d", &n);
    printf("\nEnter the cost adjacency matrix:\n");
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("Enter cost[%d][%d]: ", i, j);
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = 999;
        }
    }

    printf("The edges of Minimum Cost Spanning Tree are\n");
    while (ne < n)
```

```

{
    for (i = 1, min = 999; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            if (cost[i][j] < min)
            {
                min = cost[i][j];
                a = u = i;
                b = v = j;
            }
        }
    }
    u = find(u);
    v = find(v);
    if (uni(u, v))
    {
        printf("%d edge (%d,%d) = %d\n", ne++, a, b, min);
        mincost += min;
    }
    cost[a][b] = cost[b][a] = 999;
}

printf("\n\tMinimum cost = %d\n", mincost);
}

int find(int i)
{
    while (parent[i])
        i = parent[i];
    return i;
}

```

```

int uni(int i, int j)
{
    if (i != j)
    {
        parent[j] = i;
        return 1;
    }
    return 0;
}

```

OUTPUT:

```

PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> gcc pgm14.c -o pgm14
PS C:\Users\gmrto\OneDrive\Desktop\ads\AlbinBenny> ./pgm14

```

```

    Implementation of Kruskal's Algorithm

Enter the no. of vertices:4

Enter the cost adjacency matrix:
Enter cost[1][1]: 0
Enter cost[1][2]: 1
Enter cost[1][3]: 2
Enter cost[1][4]: 3
Enter cost[2][1]: 1
Enter cost[2][2]: 0
Enter cost[2][3]: 4
Enter cost[2][4]: 5
Enter cost[3][1]: 2
Enter cost[3][2]: 4
Enter cost[3][3]: 0
Enter cost[3][4]: 6
Enter cost[4][1]: 3
Enter cost[4][2]: 5
Enter cost[4][3]: 6
Enter cost[4][4]: 0
The edges of Minimum Cost Spanning Tree are
1 edge (1,2) = 1
2 edge (1,3) = 2
3 edge (1,4) = 3

    Minimum cost = 6

```

RESULT:

Program executed and Output obtained successfully.